InterNull: Quantum-Safe Blockchain Accounts Without Post-Quantum Primitives

Merkle-Rooted One-Time Signatures with Pre-Execution Nullifier Commitment

Jamshed Memon*
Analog Mutations Ltd, London, United Kingdom

April 2026

Abstract

The advent of cryptographically relevant quantum computers (CRQCs) poses an existential threat to blockchain systems relying on elliptic curve cryptography. Recent resource estimates published in March 2026 suggest that fewer than 500,000 superconducting physical qubits (Babbush et al., Google Quantum AI) or as few as approximately 10,000 reconfigurable neutral-atom qubits (Cain et al., Caltech/Oratomic) may suffice to solve the Elliptic Curve Discrete Logarithm Problem (ECDLP) for 256-bit curves, compressing previously anticipated timelines significantly. Conventional mitigation strategies replace ECDSA with post-quantum signature schemes (ML-DSA, SLH-DSA, FN-DSA), incurring substantial penalties in signature size (up to 47×), verification latency, and on-chain state bloat.

This paper introduces InterNull, a protocol-layer construction that achieves post-quantum security without deploying post-quantum cryptographic primitives. The base protocol composes three elements: (i) Merkle tree-based account identity, where a wallet is identified by the root of a tree containing $2^{10} = 1024$ ECDSA keypairs; (ii) one-time signature enforcement via on-chain nullifiers; and (iii) a novel *pre-execution nullifier commitment* that burns the nullifier before the signed transaction enters the network, eliminating the quantum front-running window entirely.

Two concrete instantiations are described. **InterNull-W** is a smart contract wallet deployable on Ethereum today via ERC-4337 account abstraction, using hardware-bound WebAuthn passkey signatures for non-deterministic key derivation. **InterNull-X** is a cross-chain privacy layer in which a dealer-free Distributed Key Generation (DKG) ceremony among n custodian nodes replaces the hardware enclave, producing one-time ECDSA keypairs for shielded cross-chain transfers with unlinkable deposits and withdrawals. Both instantiations reduce security to hash pre-image resistance and a trust assumption (hardware enclave integrity or custodian non-collusion, respectively) rather than the lattice or code-based hardness assumptions underlying NIST standards. A reference implementation of InterNull-W is released as a Solidity smart contract; InterNull-X is fully specified together with its DKG and mini-VSS derivation layers. An on-chain cost analysis shows transaction overhead comparable to standard ECDSA with zero additional state bloat.

Keywords: post-quantum cryptography, blockchain, Merkle signature scheme, one-time signatures, nullifier commitment, smart contract wallet, cross-chain privacy, distributed key generation, ECDSA, WebAuthn

*Corresponding author. Email: `jamshed@analogmutations.com`

1 Introduction

The security of virtually all deployed blockchain systems rests on the assumed hardness of the Elliptic Curve Discrete Logarithm Problem (ECDLP). Bitcoin, Ethereum, and Solana all employ ECDSA or EdDSA over curves of 256-bit order (secp256k1, P-256, Ed25519) for transaction authentication [1, 2]. A sufficiently powerful quantum computer running Shor's algorithm [3] would recover private keys from public keys in polynomial time, enabling unrestricted forgery.

Recent developments have dramatically compressed estimated timelines. Gidney's 2025 paper from Google Quantum AI demonstrated that RSA-2048 could be factored with fewer than one million noisy physical qubits [5], a 20× reduction from 2019 estimates [4]. In March 2026, two concurrent papers further tightened estimates for ECDLP. Babbush et al. at Google Quantum AI, together with the Ethereum Foundation and Stanford, demonstrated that ECDLP-256 could be broken with fewer than 500,000 superconducting physical qubits on conservative surface-code assumptions [6]. Independently, Cain et al. at Caltech and Oratomic showed that the same problem admits solution with as few as 10,000 reconfigurable neutral-atom qubits using a lifted-product qLDPC architecture [7]. The Chevignard, Fouque, and Schrottenloher algorithm at EUROCRYPT 2026 reduced the logical qubit requirement for P-256 to 1,193 [8]. Hardware roadmaps from IBM, Quantinuum, and PsiQuantum project systems at these scales within the late 2020s to early 2030s [9].

The standard response is to migrate to NIST-standardised post-quantum signature schemes: ML-DSA (FIPS 204) [10], SLH-DSA (FIPS 205) [11], or the forthcoming FN-DSA [12]. However, this migration imposes severe costs on blockchain systems [13]:

- **Signature size:** ML-DSA-65 signatures are 3,309 bytes versus 64 bytes for ECDSA ($\sim 52\times$ increase).
- **Public key size:** ML-DSA-65 public keys are 1,952 bytes versus 33 bytes ($\sim 59\times$ increase).
- **State bloat:** Quantum-resistant accounts require $\sim 59\times$ more persistent storage [14].
- **Throughput degradation:** Empirical benchmarks on permissioned systems show 30–60% throughput reduction [15].

Stateful hash-based signature schemes (XMSS, LMS), standardised in NIST SP 800-208 [16], offer post-quantum security with conservative assumptions but introduce the critical state management problem: signing with an already-used leaf catastrophically compromises security. NIST itself notes these schemes are "not suitable for general use" [17].

We observe that the core vulnerability in classical ECDSA on blockchains is not the algorithm itself, but the *exposure* of public keys to adversaries with unbounded time. If no public key is ever available to an adversary for longer than a single block confirmation period, and each key is provably used at most once, then even a quantum adversary with access to Shor's algorithm faces a fundamentally different, and far more constrained, attack surface.

1.1 Contributions

This paper makes the following contributions:

1. A chain-agnostic base protocol for post-quantum-secure blockchain accounts using exclusively classical cryptographic primitives (Section 3).
2. A novel *pre-execution nullifier commitment* protocol that eliminates the quantum front-running window even against malicious nodes (Section 4).
3. Two concrete instantiations: **InterNull-W**, a smart contract wallet deployable on Ethereum L1 via ERC-4337 with passkey-derived keys; and **InterNull-X**, a cross-chain privacy layer using DKG-derived one-time keys for shielded transfers (Section 5).

4. Formal security proofs under well-defined adversarial models, with security reductions for both instantiations (Section 6).
5. Reference implementation of the smart contract wallet variant together with an on-chain cost analysis (Sections 7 and 8).

2 Preliminaries

2.1 Notation

Let λ denote the security parameter. We write $x \xleftarrow{\$} S$ for uniform random sampling from set S. For a hash function $\mathcal{H} : \{0,1\}^* \to \{0,1\}^n$, we assume $n = 256$ throughout (SHA-256 or Keccak-256). We denote the secp256k1 elliptic curve group as $\mathbb{G}$ with generator G and prime order q. For a Merkle tree $\mathcal{M}$ of depth d, we write $\mathcal{M}.\mathsf{root}$ for the root hash and π_i for the authentication path of leaf i.

2.2 ECDSA

Definition 2.1 (ECDSA Signature Scheme)**.** *The Elliptic Curve Digital Signature Algorithm over secp256k1 consists of three algorithms* $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$*:*

$$\mathsf{KeyGen}(1^\lambda) : sk \xleftarrow{\$} \mathbb{Z}_q,\ pk = sk \cdot G \tag{1}$$

$$\begin{aligned}\mathsf{Sign}(sk, m) : \sigma = (r, s)\ \textit{where}\ r &= (k \cdot G)_x \bmod q,\\ s &= k^{-1}(H(m) + r \cdot sk) \bmod q\end{aligned} \tag{2}$$

$$\begin{aligned}\mathsf{Verify}(pk, m, \sigma) : \textit{Accept iff}\ &(s^{-1}H(m) \cdot G\\ &+ s^{-1}r \cdot pk)_x \equiv r \pmod q\end{aligned} \tag{3}$$

where $k \xleftarrow{\$} \mathbb{Z}_q^*$ *is a per-signature nonce.*

2.3 Merkle Trees

Definition 2.2 (Merkle Hash Tree)**.** *Given* $N = 2^d$ *leaf values* $\ell_0, \ldots, \ell_{N-1}$ *and a collision-resistant hash function* $\mathcal{H}$*, the Merkle tree* $\mathcal{M}$ *is defined recursively:*

$$\mathsf{node}_{d,i} = \ell_i, \quad \mathsf{node}_{j,i} = \mathcal{H}(\mathsf{node}_{j+1,2i} \| \mathsf{node}_{j+1,2i+1}) \tag{4}$$

The root is $\mathcal{M}.\mathsf{root} = \mathsf{node}_{0,0}$*. The authentication path for leaf* i *is the sequence of* d *sibling hashes* $\pi_i = (\mathsf{sib}_0, \ldots, \mathsf{sib}_{d-1})$ *enabling reconstruction of the root from* ℓ_i*.*

2.4 WebAuthn Passkeys and Hardware Enclaves

Definition 2.3 (Hardware-Bound Passkey Signature)**.** *A WebAuthn passkey [24] generates a credential* $(sk_{\mathsf{auth}}, pk_{\mathsf{auth}})$ *within a hardware security module (TPM, Secure Enclave, Titan). The signing operation*

$$\sigma_{\mathsf{auth}} = \mathsf{PassKey.Sign}(sk_{\mathsf{auth}}, \mathsf{challenge}) \tag{5}$$

is executed entirely within the enclave. The private key sk_{auth} *is non-exportable: no software, including the operating system, can extract* sk_{auth} *from the hardware boundary.*

2.5 Quantum Threat Model

Definition 2.4 (Quantum ECDLP Adversary)**.** *A quantum adversary $\mathcal{A}_Q$ is a polynomial-time quantum Turing machine with access to Shor's algorithm [3]. Given a public key $pk = sk \cdot G$ on secp256k1, $\mathcal{A}_Q$ can compute sk with overwhelming probability in time:*

$$T_{\mathsf{Shor}}(256) = O(n^3) \text{ quantum gates, } n = 256 \tag{6}$$

Current resource estimates place T_{Shor} at approximately 6.5×10^9 Toffoli gates on ∼1,193 logical qubits for P-256 [8], requiring physical realisation with $< 500{,}000$ noisy qubits [6].

2.6 Distributed Key Generation

Definition 2.5 (Dealer-Free Feldman DKG)**.** *A dealer-free distributed key generation protocol among n parties $P_1, \ldots, P_n$ with reconstruction threshold t produces a virtual shared secret $s \in \mathbb{Z}_q$ via the following procedure:*

1. *Each party P_i samples a random polynomial $f_i \in \mathbb{Z}_q[X]$ of degree $t-1$ with $f_i(0) = s_i$ for $s_i \xleftarrow{\$} \mathbb{Z}_q$, and publishes Feldman commitments $\{g^{a_{i,k}}\}_{k=0}^{t-1}$ to its polynomial coefficients.*
2. *Each party P_i sends share $f_i(j)$ to party P_j for all $j \neq i$, and each recipient verifies the share against the published commitments.*
3. *Each party P_j computes its combined share $s_j = \sum_{i=1}^{n} f_i(j) \bmod q$. The virtual shared secret is $s = \sum_{i=1}^{n} s_i = f(0)$ where $f = \sum_i f_i$, and the joint public key is $pk_{\mathsf{joint}} = g^s = \prod_i g^{s_i}$.*

The secret s is never materialised: no party computes s at any point in the protocol. Any subset of t parties can reconstruct s via Lagrange interpolation

$$s = \sum_{j \in S} s_j \cdot \lambda_j^S, \quad \lambda_j^S = \prod_{\ell \in S, \ell \neq j} \frac{\ell}{\ell - j} \tag{7}$$

on their combined shares $\{(j, s_j)\}_{j \in S}$ with $|S| = t$, but the protocol does not invoke this reconstruction during normal operation.

Assumption 1 (Custodian Non-Collusion)**.** *At most $t-1$ of the n custodian nodes collude. At least $n - t + 1 \geq t$ nodes remain honest and do not disclose their combined shares to any unauthorised party.*

3 Construction

3.1 System Overview

Our construction, which we term InterNull, defines a base protocol with two interchangeable key derivation backends:

1. **Identity Layer:** Wallet or shielded account identified by a Merkle root of $N = 1024$ ECDSA public keys.
2. **Derivation Layer (interchangeable):**
   - *InterNull-W:* Hardware-bound passkey signatures yield non-deterministic, device-locked chaincode.
   - *InterNull-X:* Dealer-free DKG among n custodian nodes produces a virtual master seed; per-batch child keys are derived via HMAC and fresh per-index mini-VSS, with user-side reconstruction for signing.

3. **Signing Layer:** One-time use of ECDSA leaf keys with on-chain nullifier enforcement.
4. **Nullifier Layer:** Pre-execution commitment protocol preventing key reuse even under Byzantine and quantum adversaries.

The Identity, Signing, and Nullifier layers are identical across both instantiations. Only the Derivation Layer differs, changing the trust assumption from hardware enclave integrity to custodian non-collusion.

3.2 Key Derivation from Passkey Signatures

Let chal be a wallet-specific challenge string. The chaincode is derived as:

$$\mathsf{cc} = \mathcal{H}(\mathsf{PassKey.Sign}(sk_{\mathsf{auth}}, \mathsf{chal})) \in \{0,1\}^{256} \tag{8}$$

Given the chaincode, the i-th leaf keypair is derived as:

$$sk_i = \mathcal{H}(\mathsf{cc} \| i) \bmod q, \quad pk_i = sk_i \cdot G \tag{9}$$

for $i \in \{0, 1, \ldots, N-1\}$, where $N = 1024$.

Assumption 2 (Hardware Enclave Integrity)**.** *The passkey private key sk_{auth} cannot be extracted from the hardware enclave by any polynomial-time adversary without physical access to the device and bypass of biometric/PIN authentication.*

Proposition 3.1. *Under Assumption 2 and the pre-image resistance of $\mathcal{H}$, no adversary (classical or quantum) without physical device access can derive the chaincode cc or any leaf private key sk_i.*

Proof. The chaincode cc is a hash of the passkey signature σ_{auth}. To derive cc, an adversary must either: (a) extract sk_{auth} from the enclave, contradicting Assumption 2; or (b) find a pre-image of cc under $\mathcal{H}$, requiring $O(2^{256})$ classical operations or $O(2^{128})$ under Grover's algorithm, both infeasible. Each $sk_i = \mathcal{H}(\mathsf{cc} \| i) \bmod q$ requires knowledge of cc, completing the reduction. □

3.3 Merkle-Rooted Account Identity

The leaf values of the Merkle tree are hashes of public keys:

$$\ell_i = \mathcal{H}(pk_i), \quad i \in \{0, \ldots, N-1\} \tag{10}$$

The wallet address is derived from the Merkle root:

$$\mathsf{addr} = \mathcal{H}(\mathcal{M}.\mathsf{root})_{[0:20]} \tag{11}$$

where $[\cdot]_{[0:20]}$ denotes truncation to 20 bytes (EVM address format).

Lemma 3.2 (Address Binding)**.** *Under the collision resistance of $\mathcal{H}$, the mapping from the set of N keypairs $\{(sk_i, pk_i)\}_{i=0}^{N-1}$ to the address addr is computationally binding: no efficient adversary can find a distinct set of keypairs yielding the same address.*

3.4 One-Time Signature Protocol

To sign a transaction tx using leaf index i, the user produces:

$$\sigma_{\mathsf{tx}} = (i,\ \mathsf{Sign}(sk_i, \mathsf{tx}),\ pk_i,\ \pi_i,\ \eta_i) \tag{12}$$

where π_i is the Merkle authentication path and η_i is the nullifier:

$$\eta_i = \mathcal{H}(sk_i \| i) \tag{13}$$

Definition 3.3 (Nullifier Spent Set)**.** *The on-chain state maintains a set $\mathcal{N}$ of all consumed nullifiers. A transaction is valid only if $\eta_i \notin \mathcal{N}$. Upon acceptance, η_i is added to $\mathcal{N}$.*

3.5 Key Rotation Mechanism

The final R leaves (indices $N-R,\ldots,N-1$, where $R \geq 5$) are reserved for *rotation transactions*. A rotation transaction $\mathsf{tx}_{\mathsf{rot}}$ replaces the Merkle root stored on-chain:

$$\mathsf{tx}_{\mathsf{rot}} = \big(i_{\mathsf{rot}},\ \mathsf{Sign}(sk_{i_{\mathsf{rot}}}, \mathcal{M}'.\mathsf{root}),\ pk_{i_{\mathsf{rot}}},\ \pi_{i_{\mathsf{rot}}},\ \eta_{i_{\mathsf{rot}}},\ \mathcal{M}'.\mathsf{root}\big) \tag{14}$$

where $\mathcal{M}'$ is a freshly generated Merkle tree (from a new passkey-derived chaincode or the same chaincode with a new generation counter). The address addr is preserved:

$$\mathsf{AccountState}[\mathsf{addr}].\mathsf{root} \leftarrow \mathcal{M}'.\mathsf{root} \tag{15}$$

This enables indefinite account longevity despite the finite number of leaves per tree.

4 Pre-Execution Nullifier Commitment

The critical innovation of this work is the separation of nullifier commitment from transaction execution. In prior nullifier-based designs (e.g., Zcash [22], Tornado Cash [23]), the nullifier is revealed and committed *within* the transaction execution context. This creates a window during which the signed transaction exists in the mempool with the public key exposed but the nullifier not yet committed.

4.1 Threat: Quantum Front-Running

Consider a quantum adversary $\mathcal{A}_Q$ with access to the network mempool:

1. $\mathcal{A}_Q$ observes $\sigma_{\mathsf{tx}} = (i, \sigma, pk_i, \pi_i, \eta_i)$ in the mempool.
2. $\mathcal{A}_Q$ runs Shor's algorithm to recover sk_i from pk_i.
3. $\mathcal{A}_Q$ constructs a competing transaction $\mathsf{tx}' \neq \mathsf{tx}$ with $\mathsf{Sign}(sk_i, \mathsf{tx}')$.
4. $\mathcal{A}_Q$ front-runs by submitting tx' with higher priority.

If the nullifier is committed atomically with the transaction, step (4) fails only if tx is confirmed before $\mathcal{A}_Q$ completes step (2). The security window is therefore *the block confirmation time*, which is typically 2–12 seconds.

4.2 Threat: Malicious Sequencer

In an L2 architecture with a single sequencer, a more severe attack is possible:

1. The user submits σ_{tx} to the sequencer.
2. The malicious sequencer extracts pk_i, runs Shor's algorithm (given future quantum capabilities), and recovers sk_i.
3. The sequencer *drops* the user's transaction and substitutes its own.
4. Since the nullifier was never committed (the sequencer controls inclusion), the forged transaction is accepted by all other nodes.

4.3 Protocol: Nullifier-First Architecture

We introduce a two-phase transaction protocol:

Algorithm 1 InterNull Transaction Protocol

Require: Leaf index i, transaction data tx, secret key sk_i
Ensure: Transaction included or rejected
$\eta_i \leftarrow \mathcal{H}(sk_i \| i)$ ▷ Compute nullifier
$\mathsf{msg}_\eta \leftarrow (\eta_i, \mathsf{addr}, \mathsf{nonce})$ ▷ Nullifier message
Broadcast msg_η to all validator nodes ▷ Phase 1
Wait for nullifier propagation confirmation
$\sigma \leftarrow \mathsf{Sign}(sk_i, \mathsf{tx})$ ▷ Sign transaction
$\sigma_{\mathsf{tx}} \leftarrow (i, \sigma, pk_i, \pi_i, \eta_i)$
Broadcast σ_{tx} to the network ▷ Phase 2

Definition 4.1 (Pre-Execution Nullifier Commitment)**.** *A node accepts a transaction σ_{tx} containing nullifier η_i* ***if and only if****:*

$$\eta_i \in \mathcal{N}_{\mathsf{pending}} \cup \mathcal{N}_{\mathsf{committed}} \tag{16}$$

where $\mathcal{N}_{\mathsf{pending}}$ is the set of nullifiers received in Phase 1 but not yet associated with an executed transaction, and $\mathcal{N}_{\mathsf{committed}}$ is the set of nullifiers from completed transactions.

The key invariant is:

$$\forall \sigma_{\mathsf{tx}} \text{ in mempool} : \eta(\sigma_{\mathsf{tx}}) \in \mathcal{N}_{\mathsf{pending}} \cup \mathcal{N}_{\mathsf{committed}} \tag{17}$$

A transaction referencing a nullifier not in $\mathcal{N}_{\mathsf{pending}}$ is immediately rejected.

4.4 Nullifier Gossip as Independent Transaction Type

We now describe the target architecture for decentralised nullifier propagation. We emphasise that this is a *proposed* architecture; the present InterNull-W deployment on Ethereum L1 achieves pre-execution commitment using two ordinary L1 transactions (Section 5.1), and the InterNull-X architecture operates within the sequencer trust model described in Section 4.5. Full decentralised gossip, including Byzantine-resilient propagation analysis, is left as future work; we provide here the interface and a bandwidth argument motivating its feasibility.

In the target architecture, the nullifier message msg_η is propagated as a first-class network message, independent of transaction gossip. Its compact size (approximately 96 bytes: 32-byte nullifier, 20-byte address, 32-byte nonce, 12-byte metadata) suggests that propagation completes in a small constant number of gossip rounds relative to full transaction propagation, under a standard honest-majority gossip assumption.

Proposition 4.2 (Nullifier Propagation Bandwidth Advantage)**.** *Let T_η denote the time for msg_η to reach $> 2/3$ of network nodes under an honest-majority gossip model, and T_{tx} the time for σ_{tx} under the same model. Then:*

$$T_\eta \ll T_{\mathsf{tx}} \tag{18}$$

since $|\mathsf{msg}_\eta| \approx 96$ bytes $\ll |\sigma_{\mathsf{tx}}| \approx 400$ bytes, and msg_η requires no state-transition validation.

This bandwidth argument is necessary but not sufficient for security against adaptive Byzantine adversaries (for instance, targeted eclipse attacks on the user's first-hop peers, or partial partitions of the gossip network). A complete analysis of nullifier gossip security against such adversaries is beyond the scope of this paper and is left as future work.

4.5 L2 Sequencer Architecture (Current Deployment)

In the current InterNull-X deployment, a single sequencer processes both nullifier commitments and transaction execution. The security model aligns with standard L2 trust assumptions [26]: users trust the sequencer for liveness and ordering. The pre-execution nullifier commitment is processed atomically within the sequencer:

$$\mathsf{Sequencer} : \mathsf{msg}_\eta \xrightarrow{\text{validate}} \mathcal{N}_{\mathsf{pending}} \xrightarrow{\sigma_{\text{tx}}} \mathcal{N}_{\mathsf{committed}} \tag{19}$$

This does not remove the sequencer from the trust set. It constrains the sequencer's behaviour: an honest sequencer's pre-commit enforcement prevents any external quantum adversary from front-running, and a dishonest sequencer is constrained by the same L2 trust assumptions that already apply to all L2 transactions in the ecosystem. The decentralised nullifier gossip of Section 4.4 represents the target architecture for removing the single-sequencer trust assumption in a future deployment.

5 Instantiations

The base InterNull protocol (Merkle identity, OTS enforcement, nullifier commitment) is agnostic to the key derivation mechanism. This section describes two instantiations that target distinct use cases while sharing the same post-quantum security argument.

5.1 InterNull-W: Smart Contract Wallet on Ethereum

InterNull-W targets single-chain account security and is deployable on Ethereum mainnet today as a Solidity smart contract, requiring no protocol-level modifications.

5.1.1 Key Derivation

Key derivation uses hardware-bound passkey signatures as described in Section 3.2. The chaincode $\mathsf{cc} = \mathcal{H}(\mathsf{PassKey.Sign}(sk_{\mathsf{auth}}, \mathsf{chal}))$ is computed once during wallet creation, and all N leaf keys are derived deterministically from cc via Equation (9).

5.1.2 Smart Contract Architecture

The wallet contract stores:

$$\mathsf{WalletState} = (\mathsf{root},\ \mathsf{mapping}(\eta \to \mathsf{bool}),\ \mathsf{owner}_{\mathsf{rotation}})$$

The contract exposes three functions:

(1) `commitNullifier`(η)**:** Writes η to $\mathcal{N}_{\mathsf{pending}}$. This transaction contains no public key, only a 32-byte hash, so it exposes no material for quantum attack.

(2) `execute`$(i, \sigma, pk_i, \pi_i, \eta_i, \mathsf{calldata})$**:** Verifies the Merkle proof, ECDSA signature (via `ecrecover`), and nullifier status, then executes the calldata. The nullifier must already exist in $\mathcal{N}_{\mathsf{pending}}$.

(3) `rotate`$(\mathsf{root}', i, \sigma, pk_i, \pi_i, \eta_i)$**:** Replaces the stored root using a reserved rotation leaf. The address is preserved.

5.1.3 ERC-4337 Compatibility

InterNull-W integrates with ERC-4337 account abstraction [25] by implementing the `IAccount` interface. The `validateUserOp` function performs the Merkle proof verification, ECDSA recovery, and nullifier check. The nullifier commitment can be batched as a preceding UserOperation in the same bundle, ensuring atomicity without requiring two separate on-chain transactions.

5.1.4 Trust Model

InterNull-W inherits Ethereum's full decentralised security: the nullifier commitment is confirmed by the entire validator set. No single entity can drop a nullifier commitment and front-run the user. The "malicious sequencer" attack (Section 6.4) does not apply because Ethereum L1 has no single sequencer.

5.2 InterNull-X: Cross-Chain Privacy via DKG

InterNull-X targets cross-chain private transfers. The hardware enclave is replaced by a distributed key generation ceremony among custodian nodes.

5.2.1 Key Derivation via Dealer-Free DKG with Hierarchical Derivation

InterNull-X employs a two-layer key derivation architecture.

Master layer. The n custodian nodes execute a dealer-free Feldman DKG (Definition 2.5) to produce combined shares $\{ms_j\}_{j=1}^{n}$ of a virtual master seed s, together with a shared chain code $\mathsf{cc}_{\mathsf{master}} \in \{0,1\}^{256}$ formed as the XOR of per-node random contributions. The joint master public key is $pk_{\mathsf{master}} = g^s$, computed as the sum of the first Feldman commitments published by each node. The virtual master seed $s = f(0)$ is never reconstructed.

Child layer. For each shielded batch identified by context string ctx (encoding chain, token, denomination, and batch ID), a context chain code is derived as

$$\mathsf{cc}_{\mathsf{ctx}} = \mathsf{HMAC\text{-}SHA512}(\mathsf{cc}_{\mathsf{master}}, \mathsf{ctx})[0:32]. \tag{20}$$

For each child key index $i \in \{0, \ldots, N-1\}$ in the batch, each custodian node j derives a local secret from its own master share:

$$k_{j,i} = \mathsf{HMAC\text{-}SHA512}(\mathsf{cc}_{\mathsf{ctx}}, ms_j \| i \| \mathsf{tag})[0:32] \bmod q, \tag{21}$$

where tag is a fixed domain separator (the reference implementation uses the ASCII string `"secure_v2"`). Each node j then constructs a fresh local polynomial $g_{j,i}(x) = k_{j,i} + b_{j,i,1} \cdot x + \cdots + b_{j,i,t-1} \cdot x^{t-1}$ with uniformly random coefficients $b_{j,i,\ell} \xleftarrow{\$} \mathbb{Z}_q$, and distributes evaluations $\{g_{j,i}(\ell)\}_{\ell \neq j}$ to peers over authenticated channels alongside Feldman commitments. Each node accumulates its combined child share

$$S_{j,i} = \sum_{k=1}^{n} g_{k,i}(j) \bmod q. \tag{22}$$

The child private key $sk_i = \sum_{j=1}^{n} k_{j,i}$ is not reconstructed at any custodian node. The corresponding child public key $pk_i = sk_i \cdot G$ is computed as the sum of the first Feldman commitments published by each node for index i, and the Merkle tree over the batch is constructed from $\{pk_i\}_{i=0}^{N-1}$ per Equation (10).

Retrieval and signing. To spend leaf i, the user requests combined shares from custodian nodes. Any t responses $\{(j, S_{j,i})\}$ suffice to reconstruct sk_i via Lagrange interpolation (Equation (7)). The

user then signs the withdrawal transaction locally using standard ECDSA. No custodian node ever materialises sk_i or s.

5.2.2 Cross-Chain Transfer Protocol

Deposit (Source Chain C_A):

1. User submits a deposit of value v to the InterNull-X contract on C_A.
2. The DKG nodes generate a fresh Merkle tree for this deposit, producing $\mathcal{M}.\mathsf{root}$.
3. The contract records $(\mathcal{M}.\mathsf{root}, v, \eta_{\mathsf{deposit}})$ where η_{deposit} is the deposit nullifier.

Withdrawal (Destination Chain C_B):

1. User requests withdrawal on C_B, providing the target address and proof of deposit.
2. The user retrieves combined child key shares $\{(j, S_{j,i})\}$ for leaf index i from t custodian nodes, reconstructs sk_i locally via Lagrange interpolation (Equation (7)), and produces a standard ECDSA signature $\mathsf{Sign}(sk_i, tx_{\mathsf{withdrawal}})$ on their own device.
3. The signed withdrawal transaction is submitted to C_B with the Merkle proof, nullifier, and signature.
4. The nullifier η_i is committed (pre-execution) on C_B, then the withdrawal executes.

5.2.3 Cross-Chain Unlinkability

Proposition 5.1 (Deposit-Withdrawal Unlinkability)**.** *Under the pre-image resistance of $\mathcal{H}$ and Assumption 1, an adversary observing both chain C_A and chain C_B cannot link a specific deposit on C_A to a withdrawal on C_B, provided the anonymity set contains at least two deposits of equal value.*

Proof sketch. The deposit on C_A reveals $\mathcal{M}.\mathsf{root}$ and η_{deposit}. The withdrawal on C_B reveals pk_i (a DKG-derived one-time key), π_i, and η_i. Linking requires showing that pk_i belongs to the Merkle tree committed on C_A, which requires either: (a) knowledge of $\mathsf{cc}_{\mathsf{ctx}}$ (protected by custodian non-collusion), or (b) the ability to verify the Merkle proof against $\mathcal{M}.\mathsf{root}$. However, $\mathcal{M}.\mathsf{root}$ is committed on C_A while the withdrawal occurs on C_B, and the observer would need to enumerate all possible deposit roots. With homogeneous deposit values, the linking advantage is $1/|\mathsf{AnonSet}|$. □

5.2.4 Trust Model

InterNull-X replaces Assumption 2 (hardware integrity) with Assumption 1 (custodian non-collusion). The PQ security argument is otherwise identical: each ECDSA key is used exactly once, and the nullifier is committed before the public key is exposed. Reconstructing sk_i on the user's device during signing is standard practice for ECDSA wallets and does not weaken the post-quantum argument: the public key pk_i only becomes visible on-chain after the nullifier commitment has been confirmed.

5.3 Unified Security Argument

Both instantiations share the same base security structure:

The post-quantum security in both cases derives from the *same* three properties: (1) one-time key use enforced by nullifiers, (2) key derivation protected by an "unreachable secret" (enclave or custodian cluster), and (3) pre-execution nullifier commitment eliminating the front-running window. The choice of instantiation determines the trust model but not the PQ security class.

Table 1: Trust Model Comparison Across Instantiations

Property	**InterNull-W**	**InterNull-X**
Key derivation	Passkey (enclave)	Dealer-free DKG + derivation
Trust assumption	Hardware integrity	Custodian non-collusion
Deployment	Smart contract (L1)	Cross-chain bridges
Privacy	Account-level	Transaction-level
Validator set	Ethereum (full)	Custodian nodes + chains
PQ security basis	Hash + enclave	Hash + VSS custody

6 Security Analysis

We define and analyse security against four adversarial models of increasing strength.

6.1 Security Definitions

Definition 6.1 (EU-CMA Security of InterNull)**.** *The InterNull scheme is existentially unforgeable under chosen message attack if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$:*

$$\Pr\left[\mathsf{Forge}_{\mathcal{A}}^{\mathsf{InterNull}}(\lambda) = 1\right] \leq \mathsf{negl}(\lambda) \tag{23}$$

where Forge *is the experiment in which $\mathcal{A}$ is given* addr *and oracle access to sign transactions (consuming leaves), and must produce a valid transaction σ^*_{tx} for a message tx^* not previously queried, such that:*

1. $\mathsf{Verify}(pk_j, \mathsf{tx}^*, \sigma^*) = 1$ *for some leaf j,*
2. *the Merkle proof π_j is valid for $\mathcal{M}.\mathsf{root}$, and*
3. *$\eta_j \notin \mathcal{N}$.*

6.2 Adversary 1: Classical Remote Adversary

Theorem 6.2 (Classical Unforgeability)**.** *Under the EU-CMA security of ECDSA [18] and the collision resistance of $\mathcal{H}$, no PPT classical adversary can forge a InterNull transaction.*

Proof. A classical adversary must either:

(a) Forge an ECDSA signature without knowledge of sk_i: contradicts EU-CMA of ECDSA.
(b) Find a leaf pk' and valid Merkle proof π' such that $\mathsf{Verify}_{\mathcal{M}}(\pi', pk', \mathcal{M}.\mathsf{root}) = 1$ but $pk' \neq pk_j$ for any j: contradicts collision resistance.
(c) Reuse a previously consumed leaf j: $\eta_j \in \mathcal{N}$, so the transaction is rejected.
(d) Derive sk_i from cc: requires breaking pre-image resistance of $\mathcal{H}$, costing $O(2^{256})$.
(e) Derive cc without device access: contradicts Assumption 2.

All paths lead to contradiction with standard assumptions. □

6.3 Adversary 2: Quantum Remote Adversary

Theorem 6.3 (Post-Quantum Security via One-Time Exposure)**.** *Under Assumption 2, the pre-image resistance of $\mathcal{H}$ against Grover's algorithm, and the pre-execution nullifier commitment protocol (Section 4.3), no quantum adversary $\mathcal{A}_Q$ without physical device access can forge a InterNull transaction.*

Proof. We consider the attack surface available to $\mathcal{A}_Q$:

Case 1: Derive cc. Same as classical case: requires enclave breach or hash pre-image. Grover's algorithm provides at most quadratic speedup, reducing pre-image resistance from 2^{256} to 2^{128}, which remains infeasible.

Case 2: Recover sk_i from an observed pk_i via Shor's algorithm. For a leaf j used in a confirmed transaction, the adversary observes pk_j but $\eta_j \in \mathcal{N}$, so sk_j is useless.

For unused leaves: the Merkle authentication path π_j reveals sibling *hashes* $\mathcal{H}(\mathsf{node})$, not raw public keys of other leaves. Specifically, sibling nodes at height $h > 0$ are hashes of subtrees, from which individual public keys cannot be extracted under pre-image resistance. At height $h = 1$, the sibling is $\mathcal{H}(pk_{j\oplus 1})$; recovering $pk_{j\oplus 1}$ requires inverting $\mathcal{H}$, which is infeasible.

Case 3: Quantum front-running. $\mathcal{A}_Q$ observes (pk_i, σ_i) in the mempool and attempts to:

1. Run Shor's to recover sk_i from pk_i.
2. Construct and submit a competing transaction.

By Definition 4.1, the nullifier η_i was committed in Phase 1 *before* the signed transaction (containing pk_i) entered the network. Therefore, any competing transaction using η_i is rejected (nullifier already spent), and $\mathcal{A}_Q$ cannot produce a valid nullifier for leaf i without sk_i (which would require the chaincode, see Case 1).

Case 4: Compute η_i without sk_i. The nullifier $\eta_i = \mathcal{H}(sk_i \| i)$ requires knowledge of sk_i. Attempting to forge a nullifier for an unused leaf j requires pre-image inversion of $\mathcal{H}$, infeasible even with Grover.

All cases reduce to standard assumptions. □

6.4 Adversary 3: Malicious Sequencer (L2 Model)

Theorem 6.4 (Security Against Malicious Sequencer)**.** *In the L2 model with pre-execution nullifier commitment, a malicious sequencer with quantum capabilities cannot forge transactions for accounts it does not control, provided the nullifier has been propagated to at least one honest node.*

Proof. The malicious sequencer $\mathcal{A}_S$ receives $(\mathsf{msg}_\eta, \sigma_{\mathsf{tx}})$ from the user. $\mathcal{A}_S$ has two strategies:

Strategy A: Drop transaction, forge with recovered key. $\mathcal{A}_S$ recovers sk_i from pk_i via Shor's, constructs tx', and submits $\mathsf{Sign}(sk_i, \mathsf{tx}')$ with nullifier η_i. However, η_i was already committed by the user's Phase 1 message and has propagated to other nodes. In the decentralised model (Section 4.4), honest nodes have $\eta_i \in \mathcal{N}_{\mathsf{pending}}$, associated with the user's address and nonce. The sequencer's forged transaction uses the same η_i and is therefore indistinguishable from the user's transaction at the nullifier level. But since $\mathcal{A}_S$ substitutes a different tx', honest verifiers can detect this through auxiliary mechanisms (e.g., commitment to $\mathcal{H}(\mathsf{tx})$ in Phase 1).

In the current single-sequencer L2 model, $\mathcal{A}_S$ controls transaction inclusion and can substitute tx'. This is the standard L2 trust assumption: the sequencer is trusted for ordering. Our construction does not worsen this assumption but provides a clear upgrade path via nullifier gossip.

Strategy B: Use a different leaf. $\mathcal{A}_S$ attempts to use an unused leaf $j \neq i$. This requires knowledge of sk_j, which requires the chaincode cc, which is hardware-bound (Assumption 2). □

6.5 Adversary 4: Physical Device Compromise

Assumption 3 (Explicit Exclusion)**.** *We explicitly exclude adversaries with physical access to the user's device and the ability to bypass biometric/PIN authentication. The security argument is: if the adversary has authenticated device access, they can sign arbitrary transactions directly, rendering any cryptographic mitigation moot. This applies equally to all signature schemes, including post-quantum ones.*

This is a principled threat boundary: no digital signature scheme, classical or post-quantum, can protect against an adversary who controls the signing device.

6.6 Adversary 5: Colluding Custodians (InterNull-X)

This adversary class is specific to InterNull-X. We show that an adversary controlling up to $t-1$ custodian nodes cannot forge withdrawal transactions, under Feldman VSS security and standard assumptions on HMAC-SHA512.

Theorem 6.5 (InterNull-X Unforgeability under Custodian Non-Collusion)**.** *Let $\mathcal{C} \subset \{1,\ldots,n\}$ with $|\mathcal{C}| = t-1$ denote the set of colluding custodian nodes. Under Assumption 1 (custodian non-collusion), the EU-CMA security of ECDSA [18], the pre-image resistance of $\mathcal{H}$, and the PRF security of HMAC-SHA512 under unknown key, no PPT adversary $\mathcal{A}_{\mathcal{C}}$ corrupting the nodes in $\mathcal{C}$ can forge an InterNull-X transaction except with probability* $\mathsf{negl}(\lambda)$.

Proof. We enumerate the information available to $\mathcal{A}_{\mathcal{C}}$ after the master DKG (Definition 2.5) and any number of batch generations, and show that none of this information suffices to produce a valid unused-leaf signature.

Adversary knowledge. After the master DKG, $\mathcal{A}_{\mathcal{C}}$ holds:

- Master shares $\{ms_j\}_{j\in\mathcal{C}}$ (its own combined shares of s).
- All public Feldman commitments $\{g^{a_{i,k}}\}$ published during the DKG, hence $pk_{\mathsf{master}} = g^s$.
- All per-node chain code contributions (broadcast during DKG), hence $\mathsf{cc}_{\mathsf{master}}$.

After any batch generation for context ctx, $\mathcal{A}_{\mathcal{C}}$ additionally holds:

- $\mathsf{cc}_{\mathsf{ctx}} = \mathsf{HMAC\text{-}SHA512}(\mathsf{cc}_{\mathsf{master}}, \mathsf{ctx})[0:32]$ (computable from public data).
- Local secrets $\{k_{j,i}\}_{j\in\mathcal{C},i\in\{0,\ldots,N-1\}}$ for its own nodes.
- Mini-VSS evaluations $\{g_{k,i}(j)\}_{k\in\{1,\ldots,n\},j\in\mathcal{C}}$ (sent to its nodes during the mini-VSS exchange).
- Its own combined child shares $\{S_{j,i}\}_{j\in\mathcal{C},i}$.
- The public child keys $\{pk_i\}_{i=0}^{N-1}$ and the Merkle root $\mathcal{M}.\mathsf{root}$ (both public).

What $\mathcal{A}_{\mathcal{C}}$ does not know.

(1) Honest master shares $\{ms_j\}_{j\notin\mathcal{C}}$. By the security of Feldman VSS at threshold t, the $t-1$ shares held by $\mathcal{A}_{\mathcal{C}}$ reveal no information about s or about any honest ms_j in the information-theoretic sense [37].

(2) The virtual master seed s. Reconstructing s requires t combined shares; $\mathcal{A}_{\mathcal{C}}$ has only $t-1$.

(3) Honest local secrets $\{k_{j,i}\}_{j\notin\mathcal{C},i}$. Each such $k_{j,i} = \mathsf{HMAC\text{-}SHA512}(\mathsf{cc}_{\mathsf{ctx}}, ms_j\|i\|\mathsf{tag})$ is computed under an input containing the secret ms_j. Under the PRF security of HMAC-SHA512 (specifically, the standard assumption that HMAC-SHA512 is a secure PRF when keyed with a public value and evaluated on secret-prefixed input, which in the random oracle model follows directly from the secrecy of ms_j), the outputs $k_{j,i}$ are computationally indistinguishable from uniform over $\mathbb{Z}_q$ from $\mathcal{A}_{\mathcal{C}}$'s view.

Forgery requires signing with some leaf key. For $\mathcal{A}_{\mathcal{C}}$ to produce a valid InterNull-X transaction on unused leaf i, it must produce a valid ECDSA signature $\mathsf{Sign}(sk_i, \mathsf{tx}^*)$ together with the nullifier $\eta_i = \mathcal{H}(sk_i\|i)$. The public child key for leaf i is

$$pk_i = sk_i \cdot G, \quad \text{where } sk_i = \sum_{j=1}^{n} k_{j,i} \bmod q.$$

Since sk_i is the sum over all n local secrets, and $\mathcal{A}_{\mathcal{C}}$ lacks $\{k_{j,i}\}_{j\notin\mathcal{C}}$ (by (3) above), the adversary does not know sk_i.

We reduce forgery to three base assumptions:

Path A: Recover sk_i directly. This requires recovering at least one honest $k_{j,i}$ for $j \notin \mathcal{C}$, which by (3) is infeasible unless the adversary can invert HMAC-SHA512 on an input containing the unknown secret ms_j. Under PRF security, this occurs with negligible probability.

Path B: Forge ECDSA without sk_i. This contradicts the EU-CMA security of ECDSA [18]. Note that the child public key pk_i is public (derivable from published Feldman commitments), so the adversary has the same information classical ECDSA forgery would require.

Path C: Compute $\eta_i = \mathcal{H}(sk_i \| i)$ without sk_i. This requires inverting $\mathcal{H}$ on a specific target, which by pre-image resistance of $\mathcal{H}$ (256-bit classical, 128-bit quantum under Grover) is infeasible.

All three paths reduce to standard assumptions. □

Remark 6.6 (Rushing adversary on chain code aggregation). The chain code $\mathsf{cc}_{\mathsf{master}}$ is formed as the XOR of per-node contributions broadcast during the master DKG. Without a commit-reveal phase, a rushing adversary controlling $t-1$ nodes observes the honest contributions and may choose its own contributions adversarially, biasing $\mathsf{cc}_{\mathsf{master}}$ within a $(t-1) \cdot 256$-bit subspace. We note that this does not affect Theorem 6.5: the adversary's knowledge of $\mathsf{cc}_{\mathsf{master}}$ (and therefore $\mathsf{cc}_{\mathsf{ctx}}$) is already assumed in the proof. The security of honest $k_{j,i}$ values derives from the secrecy of ms_j (guaranteed by Feldman VSS), not from the secrecy of $\mathsf{cc}_{\mathsf{master}}$. A production deployment concerned with bias-resistance of $\mathsf{cc}_{\mathsf{master}}$ itself (for example, to prevent grinding attacks against downstream batch identifiers) can add a commit-reveal phase to the chain code aggregation, following the GJKR construction [36].

6.7 Formal Security Reduction

Theorem 6.7 (Security Reduction)**.** *The InterNull scheme is post-quantum EU-CMA secure under the conjunction of:*

1. *Pre-image resistance of $\mathcal{H}$ against quantum adversaries (*128*-bit security under Grover).*
2. *Collision resistance of $\mathcal{H}$ (*128*-bit security under quantum birthday attacks).*
3. ***InterNull-W:** Hardware enclave integrity (Assumption 2), yielding EU-CMA security by Theorems 6.2 and 6.3.*

or ***InterNull-X:** Custodian non-collusion (Assumption 1) together with PRF security of HMAC-SHA512, yielding EU-CMA security by Theorem 6.5.*

*These are **minimal** assumptions: (1) and (2) are standard for any hash-based construction, and (3/4) is a trust assumption required by any key management scheme. Notably, InterNull-X does not require hardware trust, making it suitable for decentralised, server-side deployments where no hardware enclave is available.*

6.8 Comparison of Assumption Strength

Table 2 compares the cryptographic assumptions underlying InterNull against NIST post-quantum standards and classical ECDSA.

Both InterNull instantiations share the same hash-based assumption class as SLH-DSA and XMSS/LMS but eliminate the state management vulnerability of XMSS/LMS through the nullifier mechanism. InterNull-X further eliminates hardware dependency, distributing trust across a custodian cluster.

Table 2: Comparison of Security Assumptions

Scheme	**Core Assumption**	**PQ Status**
ECDSA	ECDLP hardness	Broken
ML-DSA	Module-LWE/SIS	Believed secure
SLH-DSA	Hash pre-image	Conservative
FN-DSA	NTRU lattice	Believed secure
XMSS/LMS	Hash pre-image	Conservative
InterNull-W	**Hash + enclave**	**Conservative**
InterNull-X	**Hash + VSS custody**	**Conservative**

Table 3: Transaction Size Comparison (bytes)

Component	**ECDSA**	**ML-DSA**	**XMSS**	**InterNull**
Public key	33	1,952	64	33
Signature	64	3,309	2,692	64
Merkle proof	n/a	n/a	n/a	320
Nullifier	n/a	n/a	n/a	32
Leaf index	n/a	n/a	n/a	2
Total overhead	**97**	**5,261**	**2,756**	**451**

7 Cost Analysis

7.1 On-Chain Cost Model

We model gas costs for EVM-compatible execution. Let $G_{\mathsf{calldata}} = 16$ gas/byte and $G_{\mathsf{sload}} = 2100$ gas per storage read.

InterNull transaction overhead is 451 bytes, compared to 97 bytes for plain ECDSA (4.6× increase), 5,261 bytes for ML-DSA-65 (54×), and 2,756 bytes for XMSS (28×). The Merkle proof for depth $d = 10$ is $10 \times 32 = 320$ bytes.

7.2 Verification Cost

The precompile verification consists of:

1. ECDSA verification: ∼3,000 gas (existing `ecrecover` precompile).
2. Merkle proof verification: 10 hash computations × ∼200 gas = 2,000 gas.
3. Nullifier lookup: 1 storage read = 2,100 gas.
4. Nullifier write: 1 storage write = 20,000 gas.

Total: approximately 27,100 gas versus approximately 21,000 gas for standard ECDSA transaction verification, which is approximately 29% overhead, entirely within acceptable bounds.

7.3 Key Exhaustion and Rotation Cost

InterNull-W. With $N = 1024$ leaves and $R = 5$ reserved for rotation, an InterNull-W account supports 1,019 transactions per tree generation. Assuming an illustrative usage rate of 10 transactions per day, this provides approximately 102 days of usage before rotation. Rotation cost equals one standard transaction.

Table 4: Post-Quantum Approaches for Account-Level Blockchain Security

Scheme	**Sig Size**	**PK Size**	**State**	**Trust**	**Deployable Today**
ECDSA (baseline)	64 B	33 B	Stateless	ECDLP	Yes (broken by Shor)
ML-DSA precompile (EIP-draft)	3,309 B	1,952 B	Stateless	Module-LWE/SIS	Requires hard fork
SLH-DSA (standalone)	7,856 B	32 B	Stateless	Hash pre-image	Requires hard fork
XMSS/LMS smart-contract wallet	2,692 B	64 B	**Stateful**	Hash pre-image	Smart contract
Solana Winternitz Vault [27]	∼2,144 B	32 B	Per-tx vault	Hash pre-image	Native Solana contract
InterNull-W	64 B + 354 B proof	33 B	Nullifier set	Hash + enclave	Yes (Solidity + ERC-4
InterNull-X	64 B + 354 B proof	33 B	Nullifier set	Hash + VSS custody	Yes (Solidity, dual-cha

Notes. Signature and PK sizes for ML-DSA-65 per FIPS 204 [10]; SLH-DSA-128s per FIPS 205 [11]; XMSS from the standard authentication path for depth-10 trees. Solana Winternitz Vault signature size estimated from WOTS with $w = 16$ on Keccak-256. InterNull "signature" column includes the 64-byte ECDSA signature plus the $10 \times 32 + 32 + 2 = 354$-byte Merkle proof + nullifier + index overhead.

InterNull-X. The cross-chain variant follows a different key-consumption model: each deposit instantiates a fresh Merkle tree sized to the number of withdrawals that deposit will require. Tree capacity is therefore a per-deposit design parameter rather than a longevity constraint. For bearer-style fixed-denomination deposits, N corresponds to the number of partial withdrawals permitted from that deposit, and no tree-level rotation is required.

7.4 Comparison with Alternative PQ Approaches

We situate InterNull against three classes of post-quantum construction for blockchains: protocol-level precompile additions (e.g., ML-DSA precompile proposals), stateful hash-based wallet designs (XMSS/LMS smart-contract wallets), and Winternitz one-time signature vaults (Solana). Table 4 summarises the comparison.

Versus ML-DSA precompile proposals. Proposed EIPs for adding a native ML-DSA verification precompile to the EVM would enable lattice-based signatures, but require a hard fork and impose ∼52× signature bloat and ∼59× public key bloat per account [15]. InterNull-W delivers equivalent post-quantum security via smart contract deployment today, with transaction overhead an order of magnitude smaller (451 bytes vs 5,261 bytes).

Versus XMSS/LMS wallet constructions. Stateful hash-based signatures provide conservative post-quantum security [16] but impose a critical state-management burden: reusing a leaf index catastrophically breaks security, a hazard NIST explicitly flags as making these schemes "not suitable for general use" [17]. InterNull's nullifier mechanism provides on-chain enforcement of one-time use, eliminating the state management hazard. The trade-off is that InterNull requires a two-phase transaction flow (commit, then execute), whereas XMSS/LMS wallets are single-transaction.

Versus Solana Winternitz Vault. Solana's Winternitz Vault [27] achieves one-time-use security with a simpler trust model, relying solely on hash pre-image resistance. Each WOTS signature reveals approximately half the private key, requiring vault closure and recreation per transaction. InterNull trades a stronger trust assumption (hardware enclave for InterNull-W, or VSS custody for InterNull-X) for (a) standard ECDSA tooling compatibility (signing code path is unmodified), (b) much smaller signatures (64 B vs ∼2 KB), and (c) in the case of InterNull-X, a cross-chain privacy architecture that Winternitz vaults do not attempt. Users or applications that do not require EVM tooling compatibility or cross-chain transfer may reasonably prefer the simpler trust model of WOTS-based vaults; InterNull is aimed at use cases where ECDSA compatibility and/or

cross-chain operation are first-class requirements.

8 Implementation

8.1 InterNull-W: Solidity Smart Contract Wallet

InterNull-W is implemented as a Solidity smart contract deployable on any EVM-compatible chain without protocol modifications. The contract implements the `IAccount` interface for ERC-4337 compatibility.

8.1.1 Contract Interface

The wallet contract stores the Merkle root and nullifier set:

$$\mathsf{Storage} = (\mathsf{root} : \mathsf{bytes32},\ \mathcal{N} : \mathsf{mapping}(\mathsf{bytes32} \Rightarrow \mathsf{bool}))$$

The verification procedure within `execute` is:

Algorithm 2 InterNull-W Smart Contract Verification

Require: $(\mathsf{root}, i, \sigma, pk_i, \pi_i, \eta_i, \mathsf{calldata})$
Ensure: Execute or Revert
if $|\pi_i| \neq d$ **then revert** ▷ Proof depth check
end if
if $i \geq N$ **then revert** ▷ Index bounds check
end if
if $\mathcal{N}[\eta_i] = \mathsf{true}$ **then revert** ▷ Double-spend
end if
if $\mathsf{ecrecover}(\mathsf{txHash}, \sigma) \neq pk_i$ **then revert**
end if
$\ell \leftarrow \mathsf{keccak256}(pk_i)$
for $j = 0$ to $d - 1$ **do** ▷ Merkle verification
    if bit j of i is 0 **then** $\ell \leftarrow \mathsf{keccak256}(\ell \| \pi_i[j])$
    else $\ell \leftarrow \mathsf{keccak256}(\pi_i[j] \| \ell)$
    end if
end for
if $\ell \neq \mathsf{root}$ **then revert**
end if
$\mathcal{N}[\eta_i] \leftarrow \mathsf{true}$
execute calldata ▷ Forward call

The two-phase nullifier commitment on Ethereum L1 operates as two transactions: (1) `commitNullifier(`η_i`)` writes the nullifier to $\mathcal{N}$ without exposing any public key; (2) `execute(...)` verifies the signature and Merkle proof, requiring η_i to already be committed. Since (1) contains only a 32-byte hash, no material is available for quantum attack. The public key appears only in (2), by which time the nullifier is already confirmed on-chain by Ethereum's full validator set.

8.2 InterNull-X: Cross-Chain Privacy Layer

InterNull-X is implemented as a pair of smart contracts (deposit contract on source chain, withdrawal contract on destination chain) coordinated by a set of n custodian nodes. The custodian nodes execute a dealer-free Feldman DKG (Definition 2.5) to establish a virtual master seed that no single node learns, and derive per-batch child key shares via HMAC-derived local secrets combined through fresh per-index mini-VSS polynomials (Equations (21) and (22)). The user reconstructs each withdrawal key sk_i locally from t combined shares and signs with standard ECDSA. The deposit and withdrawal contracts implement the same Merkle+nullifier verification logic as InterNull-W, with the addition of cross-chain commitment relaying.

8.3 Nullifier Storage Optimisation

The nullifier set $\mathcal{N}$ is implemented as a Bloom filter for fast negative lookups, backed by a persistent mapping $\mathsf{mapping}(\eta_i \to \mathsf{bool})$ for definitive membership. The Bloom filter has false positive rate $p = 2^{-20}$ with $k = 14$ hash functions and $m = 20 \cdot |\mathcal{N}|$ bits of storage.

8.4 Client-Side Wallet Architecture

The wallet application manages:

- **Leaf index counter:** monotonically increasing, persisted locally and synchronised with on-chain nullifier state.
- **Passkey interaction:** single biometric prompt on wallet creation; chaincode cached in secure storage.
- **Two-phase submission:** nullifier broadcast followed by transaction broadcast, with configurable propagation delay.

9 Discussion

9.1 Relationship to Hash-Based Signatures

InterNull is structurally related to the Merkle Signature Scheme (MSS) [19], XMSS [20], and LMS [21]. The key distinction is that InterNull uses ECDSA at the leaves rather than a hash-based OTS (Winternitz or Lamport). This means the signing and verification operations are identical to standard ECDSA, and no new cryptographic primitives are introduced into the critical path. The post-quantum security comes from the *protocol layer* (one-time use + nullifier commitment + unreachable derivation secret) rather than the *algorithmic layer*.

This decomposition has practical advantages: existing tooling (hardware wallets, HSMs, wallet SDKs, threshold signing libraries) requires zero modification to the signing code path.

9.2 Smart Contract Wallet as Primary Deployment

The InterNull-W smart contract wallet is the most immediately impactful instantiation because it requires no protocol changes, no custom chain, and no new infrastructure. Any Ethereum user can deploy the contract today and achieve post-quantum withdrawal security. This contrasts sharply with proposals requiring hard forks, new signature precompiles, or chain-wide migration [34, 14].

The ERC-4337 integration further reduces adoption friction: bundlers, paymasters, and the existing account abstraction ecosystem work unchanged. The only modification is the validation logic inside the smart account.

9.3 Cross-Chain Privacy as a Use Case

InterNull-X formalises the cryptographic core of the cross-chain privacy layer. The DKG-based key derivation replaces hardware trust with custodian non-collusion trust, enabling server-side deployments where no hardware enclave is available. The one-time ECDSA construction provides the same PQ security guarantees while simultaneously delivering cross-chain unlinkability, a property not achievable with single-chain wallet constructions.

The composition of DKG + OTS + nullifiers provides a unique combination: cross-chain privacy (from DKG-derived unlinkable keys), post-quantum security (from one-time use + nullifier commitment), and standard EVM compatibility (from ECDSA signatures verifiable by existing contracts). This positions InterNull-X as a privacy layer that is simultaneously quantum-resistant without requiring ZK-SNARKs or post-quantum signature schemes on-chain.

9.4 Comparison with Ethereum's PQ Roadmap

The Ethereum Foundation has initiated research into post-quantum migration, including account abstraction (ERC-4337) [25] as a pathway for quantum-resistant signature verification. InterNull-W is fully compatible with this approach and can serve as an interim solution while protocol-level PQ migration proceeds. InterNull-X extends the approach to cross-chain scenarios that Ethereum's single-chain roadmap does not address.

9.5 Limitations

1. **Finite transaction capacity (InterNull-W):** Each tree generation supports approximately 1,019 transactions. High-frequency accounts require more frequent rotation. InterNull-X sizes trees per deposit and is not subject to this limitation.
2. **State synchronisation:** The leaf index counter must be synchronised between client and chain. Crash recovery requires scanning the on-chain nullifier set.
3. **InterNull-X liveness:** The custodian nodes must be available to serve share requests. If fewer than t nodes are online, withdrawals are blocked until quorum is restored.
4. **InterNull-W device dependency:** If the passkey device is lost and the passkey is not synced, access to unused leaves is permanently lost. Rotation must be initiated before device loss.
5. **Two-transaction overhead (InterNull-W on L1):** The nullifier commitment requires a separate on-chain transaction, adding one block of latency and gas for the commit step.

9.6 Future Work

Zero-knowledge enhancement. A ZK-STARK wrapper could prove ECDSA verification and Merkle membership without revealing either, providing information-theoretic privacy. However, as demonstrated in this paper, such overhead is *unnecessary* for post-quantum security alone.

Batched nullifier commitments. Multiple nullifiers can be committed in a single transaction via a Merkle root of pending nullifiers, amortising the gas cost of the commit phase across many operations.

Increased tree capacity. Using depth $d = 20$ yields $2^{20} = 1{,}048{,}576$ leaves, supporting decades of high-frequency use. The Merkle proof grows linearly ($20 \times 32 = 640$ bytes), a modest increase.

Hybrid DKG + passkey derivation. A construction combining both backends (passkey for user-facing authentication, DKG for cross-chain operations) could provide the strongest trust

model: an adversary would need to compromise *both* the hardware enclave and the custodian cluster.

10 Related Work

Post-quantum blockchain migration. Yang et al. [13] provide a comprehensive survey of post-quantum and quantum blockchain approaches, cataloguing lattice-based, hash-based, and code-based alternatives. Juaristi et al. [15] benchmark PQC schemes in Ethereum-based blockchains, finding significant throughput impacts. The hybrid signature approach of [14] proposes combining classical and PQ signatures at the protocol level but incurs combined overhead.

Quantum threat estimation. Gidney and Ekerå [4] established the 20-million-qubit baseline for RSA-2048. Gidney's 2025 update [5] reduced this to under one million. Babbush et al. [6] extended the analysis to ECDLP-256, estimating $< 500{,}000$ physical qubits. The Chevignard–Fouque–Schrottenloher algorithm [8] achieved 1,193 logical qubits for P-256 at EUROCRYPT 2026.

Nullifier-based privacy. Zcash [22] introduced nullifiers for preventing double-spending of shielded notes. Tornado Cash [23] adapted the concept for Ethereum-based mixing. Both use nullifiers for privacy rather than post-quantum security, and commit nullifiers within transaction execution rather than pre-execution.

Stateful hash-based signatures. NIST SP 800-208 [16] standardises XMSS and LMS. Kelsey et al. [29] address the state management vulnerability by converting HBS into threshold signature schemes. Our approach eliminates the state management problem entirely through on-chain nullifier tracking.

Solana Winternitz Vault. Little [27] implements a Winternitz OTS vault on Solana with Keccak-256, achieving 224-bit quantum preimage resistance. Each transaction requires vault closure and recreation. InterNull achieves similar guarantees with smaller signatures and standard ECDSA tooling. Project Eleven's collaboration with Solana on testnet PQ signatures [28] explores broader integration.

Hardware-bound key derivation. The WebAuthn standard [24] and FIDO2 framework provide the foundation for passkey-based authentication. Combining hardware-bound secrets with blockchain key derivation (InterNull-W) is, to our knowledge, novel in this construction.

Dealer-free DKG and Verifiable Secret Sharing. Feldman [37] introduced non-interactive verifiable secret sharing, enabling recipients to verify that dealt shares are consistent with a committed polynomial. Pedersen [35] introduced dealer-free distributed key generation via aggregation of per-party verifiable secret sharings, producing a shared secret that no single party ever learns. Combining dealer-free Feldman DKG with hierarchical HMAC-based derivation and fresh per-index mini-VSS polynomials, as in InterNull-X, enables custodial generation of arbitrarily many one-time ECDSA keypairs without any node materialising the master seed or any child private key. This construction is distinct from threshold ECDSA, which produces signatures cooperatively without key reconstruction; InterNull-X instead relies on user-side reconstruction of one-time keys and on-chain nullifier enforcement to obtain post-quantum security.

Cross-chain privacy. Zcash [22] and Tornado Cash [23] provide single-chain privacy via nullifier-based double-spend prevention. Cross-chain privacy protocols typically rely on ZK proofs for state verification across chains. InterNull-X achieves cross-chain unlinkability through DKG-derived one-time keys without requiring ZK proofs on the critical path, reducing verification cost.

11 Conclusion

We have presented InterNull, a protocol-layer construction achieving post-quantum security for blockchain systems using exclusively classical cryptographic primitives. The base protocol composes Merkle tree-based identity, one-time ECDSA with on-chain nullifier enforcement, and a novel pre-execution nullifier commitment protocol. The paper describes two instantiations: InterNull-W, a smart contract wallet deployable on Ethereum today via ERC-4337 with hardware-bound passkey key derivation; and InterNull-X, a cross-chain privacy layer using DKG-derived one-time keys for shielded transfers with deposit-withdrawal unlinkability.

The central insight is that post-quantum security does not require post-quantum algorithms when the protocol layer guarantees that: (1) each signing key is used at most once; (2) key material is derived from an unreachable secret (hardware enclave or custodian cluster); and (3) nullifiers are committed before public keys are exposed to the network.

Both instantiations reduce security to hash pre-image resistance and a trust assumption (hardware integrity or custodian non-collusion) rather than the lattice or code-based conjectures underlying NIST PQ standards. The smart contract wallet achieves this with approximately 29% gas overhead versus standard ECDSA and roughly 10× less state bloat compared to ML-DSA alternatives, while the cross-chain variant adds unlinkability at no additional cryptographic cost.

As quantum computing resource estimates continue to decrease [5, 6, 8], the urgency of deploying quantum-resistant solutions grows. InterNull offers a pragmatic, immediately deployable path, requiring no hard forks, no new precompiles, and no protocol-level changes, while providing equivalent or stronger security guarantees against foreseeable quantum adversaries. The dual-instantiation architecture demonstrates that the same base protocol can serve both single-chain account security and cross-chain privacy, unifying two previously distinct problem domains under a single post-quantum security framework.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: `https://bitcoin.org/bitcoin.pdf`

[2] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, 2014.

[3] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, Oct. 1997. DOI: 10.1137/S0097539795293172.

[4] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, Apr. 2021. DOI: 10.22331/q-2021-04-15-433. arXiv:1905.09749.

[5] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," arXiv preprint arXiv:2505.15917, May 2025. [Online]. Available: `https://arxiv.org/abs/2505.15917`

[6] R. Babbush, A. Zalcman, C. Gidney, M. Broughton, T. Khattar, H. Neven, T. Bergamaschi, J. Drake, and D. Boneh, "Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations," Google Quantum AI Whitepaper, Mar. 2026. arXiv:2603.28846. IACR ePrint 2026/625.

[7] M. Cain, Q. Xu, R. King, L. R. B. Picard, H. Levine, M. Endres, J. Preskill, H.-Y. Huang, and D. Bluvstein, "Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits," arXiv preprint arXiv:2603.28627, Mar. 2026.

[8] C. Chevignard, P.-A. Fouque, and A. Schrottenloher, "Reducing the number of qubits in quantum discrete logarithms on elliptic curves," in *Proc. EUROCRYPT 2026*, Springer, 2026. IACR ePrint 2026/280. [Online]. Available: `https://eprint.iacr.org/2026/280`

[9] IBM Quantum, "IBM Quantum development & innovation roadmap," 2025. [Online]. Available: `https://www.ibm.com/quantum/technology`

[10] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard," U.S. Department of Commerce, Aug. 2024. DOI: 10.6028/NIST.FIPS.204.

[11] National Institute of Standards and Technology, "FIPS 205: Stateless Hash-Based Digital Signature Standard," U.S. Department of Commerce, Aug. 2024. DOI: 10.6028/NIST.FIPS.205.

[12] National Institute of Standards and Technology, "FIPS 206 (Draft): FFI-Based Lattice Digital Signature Standard (FN-DSA)," 2025.

[13] Z. Yang, H. Alfauri, B. Farkiani, R. Jain, R. Di Pietro, and A. Erbad, "A survey and comparison of post-quantum and quantum blockchains," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 967–1002, 2024.

[14] C. Abrusci *et al.*, "Hybrid post-quantum signatures for Bitcoin and Ethereum: A protocol-level integration strategy," Preprints.org, Sep. 2025. [Online]. Available: `https://www.preprints.org/manuscript/202509.2079/v1`

[15] P. Juaristi, I. Agudo, R. Rios, and L. Ricci, "Benchmarking post-quantum cryptography in Ethereum-based blockchains," in *ESORICS 2024 Int. Workshops*, Springer, 2025, pp. 340–353.

[16] National Institute of Standards and Technology, "SP 800-208: Recommendation for stateful hash-based signature schemes," October 2020.

[17] NIST CSRC, "Stateful hash-based signatures project page," 2025. [Online]. Available: `https://csrc.nist.gov/projects/stateful-hash-based-signatures`

[18] D. R. L. Brown, "Generic groups, collision resistance, and ECDSA," *Des. Codes Cryptogr.*, vol. 35, no. 1, pp. 119–152, Apr. 2005. DOI: 10.1007/s10623-003-6154-z.

[19] R. C. Merkle, "A certified digital signature," in *Proc. CRYPTO '89*, Springer, 1989, pp. 218–238.

[20] A. Hülsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen, "XMSS: eXtended Merkle signature scheme," RFC 8391, IRTF, May 2018. DOI: 10.17487/RFC8391.

[21] D. McGrew, M. Curcio, and S. Fluhrer, "Leighton-Micali hash-based signatures," RFC 8554, IRTF, Apr. 2019. DOI: 10.17487/RFC8554.

[22] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from Bitcoin," in *Proc. 2014 IEEE Symp. Security & Privacy*, 2014, pp. 459–474. DOI: 10.1109/SP.2014.36.

[23] A. Pertsev, R. Semenov, and R. Storm, "Tornado Cash privacy solution version 1.4," Whitepaper, Dec. 2019. [Online]. Available: `https://berkeley-defi.github.io/assets/material/Tornado%20Cash%20Whitepaper.pdf`

[24] J. Hodges, J. C. Jones, M. B. Jones, A. Kumar, and E. Lundberg (Eds.), "Web Authentication: An API for accessing public key credentials – Level 2," W3C Recommendation, Apr. 2021. [Online]. Available: `https://www.w3.org/TR/webauthn-2/`

[25] V. Buterin, Y. Weiss, D. Tirosh, S. Nacson, A. Forshtat, K. Namber, and T. Hetzel, "ERC-4337: Account abstraction using alt mempool," Ethereum Improvement Proposal, 2021. [Online]. Available: `https://eips.ethereum.org/EIPS/eip-4337`

[26] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the security and performance of proof of work blockchains," in *Proc. 2016 ACM SIGSAC Conf. Computer and Communications Security*, 2016, pp. 3–16. DOI: 10.1145/2976749.2978341.

[27] D. Little, "Solana Winternitz Vault: Quantum-resistant lamports vault," GitHub, January 2025. [Online]. Available: `https://github.com/deanmlittle/solana-winternitz-vault`

[28] Project Eleven, "Post-quantum blockchain research initiatives," 2025. [Online]. Available: `https://www.projecteleven.com/`

[29] J. Kelsey, S. Lucks, and N. Lang, "Turning hash-based signatures into distributed signatures and threshold signatures," *IACR Commun. Cryptol.*, 2025. DOI: 10.62056/a6ksudy6b.

[30] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219. DOI: 10.1145/237814.237866.

[31] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, "Quantum resource estimates for computing elliptic curve discrete logarithms," in *Advances in Cryptology: ASIACRYPT 2017*, LNCS vol. 10625, Springer, 2017, pp. 241–270. DOI: 10.1007/978-3-319-70697-9_9.

[32] O. Regev, "An efficient quantum factoring algorithm," arXiv preprint arXiv:2308.06572, Aug. 2023. J. ACM, vol. 72, no. 1, pp. 1–13, Jan. 2025.

[33] S. Ragavan and V. Vaikuntanathan, "Space-efficient and noise-robust quantum factoring," in *Advances in Cryptology: CRYPTO 2024*, Springer, 2024. arXiv:2310.00899.

[34] V. Buterin, "Possible futures of the Ethereum protocol, part 6: The Splurge," Ethereum Foundation Blog, 2024. [Online]. Available: `https://vitalik.eth.limo/general/2024/10/29/futures6.html`

[35] T. P. Pedersen, "A threshold cryptosystem without a trusted party," in *Advances in Cryptology: EUROCRYPT '91*, LNCS vol. 547, Springer, 1991, pp. 522–526. DOI: 10.1007/3-540-46416-6_47.

[36] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," *Journal of Cryptology*, vol. 20, no. 1, pp. 51–83, January 2007. DOI: 10.1007/s00145-006-0347-3.

[37] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *Proc. 28th IEEE FOCS*, 1987, pp. 427–438.

[38] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. CRYPTO '91*, Springer, 1991, pp. 129–140.